

# Python Interview Questions for Data Analysts

## Introduction

This comprehensive guide contains the most commonly asked Python interview questions in data analyst roles. The questions cover essential topics including Pandas, NumPy, data manipulation, cleaning, and analysis—all critical skills for data analyst interviews at major tech companies and organizations.

## Interview Questions and Answers

### 1. What is Pandas, and why is it popular in data analysis?

**Answer:**

Pandas is an open-source Python library designed for data manipulation and analysis. It provides two primary data structures:

- **Series:** 1D labeled array (similar to a single column)
- **DataFrame:** 2D labeled table (similar to a spreadsheet or SQL table)

**Why it's popular:**

- Easy-to-use API for data cleaning and transformation
- Handles missing data efficiently
- Supports multiple data types (numeric, categorical, datetime)
- Integrates well with other libraries (NumPy, Matplotlib, Scikit-learn)
- Optimized performance for large datasets
- Supports reading/writing various file formats (CSV, Excel, JSON, SQL databases)

**Example:**

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.head())
```

### 2. What is NumPy, and how is it different from Python lists?

**Answer:**

NumPy (Numerical Python) is a library for numerical computing that provides support for large, multi-dimensional arrays and matrices, along with mathematical functions to operate on them efficiently.

**Key Differences:**

| Feature     | Python Lists                | NumPy Arrays                   |
|-------------|-----------------------------|--------------------------------|
| Type        | Homogeneous or mixed        | Homogeneous only               |
| Performance | Slower                      | Much faster (C implementation) |
| Memory      | Uses more memory            | Memory efficient               |
| Operations  | Element-wise requires loops | Vectorized operations          |

| Feature                       | Python Lists  | NumPy Arrays                           |
|-------------------------------|---------------|----------------------------------------|
| <b>Ease of Use</b>            | Simple syntax | Requires learning NumPy functions      |
| <b>Mathematical Functions</b> | Limited       | Extensive (linear algebra, statistics) |

#### Example:

```
import numpy as np

# Python list
python_list = [1, 2, 3, 4, 5]
result_list = [x * 2 for x in python_list] # Requires loop

# NumPy array
numpy_array = np.array([1, 2, 3, 4, 5])
result_array = numpy_array * 2 # Vectorized operation (faster)
```

### 3. How do you read a CSV file into a Pandas DataFrame?

#### Answer:

You can read a CSV file using the `pd.read_csv()` function.

#### Syntax:

```
import pandas as pd

# Basic usage
df = pd.read_csv('file.csv')

# With additional parameters
df = pd.read_csv('file.csv',
                 sep=',',           # Delimiter
                 header=0,          # Row number for column names
                 dtype={'col1': int}, # Specify data types
                 nrows=100)         # Read only first 100 rows
```

#### Common Parameters:

- `sep`: Delimiter used in the file (default: ',')
- `header`: Row number to use as column names (default: 0)
- `dtype`: Dictionary specifying data types for columns
- `na_values`: Additional strings to recognize as NaN
- `nrows`: Number of rows to read
- `skiprows`: Rows to skip

### 4. What is the difference between `loc` and `iloc` in Pandas?

#### Answer:

Both `loc` and `iloc` are used for indexing, but they work differently:

| Feature               | <code>loc</code> | <code>iloc</code>      |
|-----------------------|------------------|------------------------|
| <b>Selection Type</b> | Label-based      | Integer position-based |

| Feature         | loc                              | iloc                             |
|-----------------|----------------------------------|----------------------------------|
| <b>Syntax</b>   | loc[row_label, col_label]        | iloc[row_position, col_position] |
| <b>Includes</b> | Both start and end are inclusive | Start inclusive, end exclusive   |
| <b>Use Case</b> | When you know the label          | When you know the position       |

### Examples:

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35]
}, index=['A', 'B', 'C'])

# Using loc (label-based)
print(df.loc['A', 'Name'])      # Output: Alice
print(df.loc['A':'B', 'Age'])   # Both A and B included

# Using iloc (position-based)
print(df.iloc[0, 0])            # Output: Alice
print(df.iloc[0:2, 1])          # Rows 0-1 (2 excluded), column 1
```

## 5. How do you handle missing values in Pandas?

### Answer:

There are several methods to handle missing values (NaN) in Pandas:

#### 1. Remove missing values:

```
import pandas as pd

df.dropna()                # Remove rows with any NaN
df.dropna(subset=['col1']) # Remove rows with NaN in specific column
df.dropna(axis=1)          # Remove columns with any NaN
```

#### 2. Fill missing values:

```
df.fillna(0)                # Fill with a specific value
df.fillna(df.mean())        # Fill with mean (numeric columns)
df.fillna(method='ffill')   # Forward fill (use previous value)
df.fillna(method='bfill')   # Backward fill (use next value)
```

#### 3. Interpolate missing values:

```
df.interpolate()            # Linear interpolation
df.interpolate(method='bfill') # Backward interpolation
```

#### 4. Check for missing values:

```
df.isnull()                # Returns boolean DataFrame
df.isnull().sum()          # Count missing values per column
```

## 6. Explain the concept of broadcasting in NumPy.

### Answer:

Broadcasting is NumPy's mechanism for performing operations on arrays of different shapes. NumPy automatically expands smaller arrays to match the shape of larger arrays without copying data, making operations efficient.

### Broadcasting Rules:

1. If arrays have different numbers of dimensions, pad the smaller one with ones on the left
2. Arrays are compatible if dimensions are equal or one is 1
3. The array with dimension 1 is stretched to match the larger dimension

### Examples:

```
import numpy as np

# Example 1: 1D array + scalar
arr = np.array([1, 2, 3, 4])
result = arr + 10          # Output: [11, 12, 13, 14]

# Example 2: 2D + 1D broadcasting
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]]) # Shape: (2, 3)
vector = np.array([1, 2, 3])   # Shape: (3,)
result = matrix + vector       # Vector broadcasted to (2, 3)
# Output: [[2, 4, 6], [5, 7, 9]]

# Example 3: (2, 1) + (2, 3) broadcasting
col = np.array([[10], [20]])   # Shape: (2, 1)
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]]) # Shape: (2, 3)
result = col + matrix          # col broadcasted to (2, 3)
```

## 7. How do you filter rows in a DataFrame based on a condition?

### Answer:

You can filter DataFrame rows using boolean indexing in Pandas.

### Method 1: Single condition

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'Salary': [50000, 60000, 75000]
})

# Filter rows where Age > 25
filtered_df = df[df['Age'] > 25]

# Filter rows where Name == 'Bob'
filtered_df = df[df['Name'] == 'Bob']
```

### Method 2: Multiple conditions (AND)

```
# Age > 25 AND Salary > 55000
filtered_df = df[(df['Age'] > 25) & (df['Salary'] > 55000)]
```

### Method 3: Multiple conditions (OR)

```
# Age > 30 OR Salary > 70000
filtered_df = df[(df['Age'] > 30) | (df['Salary'] > 70000)]
```

### Method 4: Using isin() for multiple values

```
# Filter where Name is either Alice or Charlie
filtered_df = df[df['Name'].isin(['Alice', 'Charlie'])]
```

### Method 5: Using loc with conditions

```
filtered_df = df.loc[df['Age'] > 25, ['Name', 'Age']]
```

## 8. How do you merge (join) two DataFrames in Pandas?

### Answer:

Pandas provides `merge()` function to combine DataFrames based on a common column or index.

### Syntax:

```
import pandas as pd

df1 = pd.DataFrame({
    'ID': [1, 2, 3],
    'Name': ['Alice', 'Bob', 'Charlie']
})

df2 = pd.DataFrame({
    'ID': [1, 2, 3],
    'Salary': [50000, 60000, 75000]
})

# Inner join (default)
merged_df = pd.merge(df1, df2, on='ID')

# Left join
merged_df = pd.merge(df1, df2, on='ID', how='left')

# Right join
merged_df = pd.merge(df1, df2, on='ID', how='right')

# Outer join
merged_df = pd.merge(df1, df2, on='ID', how='outer')
```

### Join Types:

| Join Type    | Description                                            |
|--------------|--------------------------------------------------------|
| <b>inner</b> | Only matching rows from both DataFrames                |
| <b>left</b>  | All rows from left DataFrame, matching rows from right |
| <b>right</b> | All rows from right DataFrame, matching rows from left |
| <b>outer</b> | All rows from both DataFrames                          |

## 9. Explain the `groupby()` function in Pandas.

### Answer:

`groupby()` is used to group rows by one or more columns and apply aggregate functions to each group.

### Basic Syntax:

```
import pandas as pd

df = pd.DataFrame({
    'Department': ['IT', 'IT', 'HR', 'HR', 'Sales'],
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'Salary': [50000, 60000, 45000, 55000, 40000]
})

# Group by Department and calculate mean salary
grouped = df.groupby('Department')['Salary'].mean()
# Output:
# Department
# HR      50000.0
# IT      55000.0
# Sales   40000.0

# Multiple aggregations
grouped = df.groupby('Department').agg({
    'Salary': ['mean', 'sum', 'count'],
    'Name': 'count'
})

# Multiple grouping columns
grouped = df.groupby(['Department', 'Name'])['Salary'].sum()
```

### Common Aggregation Functions:

- `sum()`: Sum of values
- `mean()`: Average
- `median()`: Median
- `min()`: Minimum value
- `max()`: Maximum value
- `count()`: Count of values
- `std()`: Standard deviation
- `var()`: Variance

## 10. How do you handle duplicate rows in a DataFrame?

### Answer:

Duplicate rows can be identified and removed using the `duplicated()` and `drop_duplicates()` functions.

### Identifying duplicates:

```
import pandas as pd

df = pd.DataFrame({
    'ID': [1, 2, 2, 3, 3, 3],
    'Name': ['Alice', 'Bob', 'Bob', 'Charlie', 'Charlie', 'Charlie']
})
```

```
# Check for duplicates
print(df.duplicated()) # Returns boolean Series
print(df.duplicated(subset=['ID'])) # Check duplicates in specific column
```

### Removing duplicates:

```
# Remove all duplicate rows
df_no_dups = df.drop_duplicates()

# Remove duplicates based on specific columns
df_no_dups = df.drop_duplicates(subset=['ID'], keep='first')

# Remove in-place
df.drop_duplicates(inplace=True)
```

### Parameters:

- subset: Column(s) to consider for identifying duplicates
- keep: Which duplicates to keep ('first', 'last', or False for none)
- inplace: Whether to modify the original DataFrame

## 11. What is the purpose of the apply() function in Pandas?

### Answer:

apply() applies a function to each row, column, or element of a DataFrame or Series.

### Using apply() on columns:

```
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
})

# Apply function to each column
result = df.apply(sum) # Sums each column

# Apply custom function
result = df.apply(lambda x: x.max() - x.min()) # Range for each column
```

### Using apply() on rows:

```
# Apply function to each row (axis=1)
df['Total'] = df.apply(lambda row: row['A'] + row['B'], axis=1)

# Create new column with conditional logic
df['Status'] = df.apply(lambda row: 'High' if row['A'] > 2 else 'Low', axis=1)
```

### Using apply() on Series:

```
# Apply to a Series
series = pd.Series([1, 2, 3, 4, 5])
result = series.apply(lambda x: x ** 2) # Square each value
```

## 12. How do you rename columns in a DataFrame?

### Answer:

Columns can be renamed using the `rename()` method or by directly assigning to `columns` attribute.

#### Method 1: Using `rename()` with a dictionary

```
import pandas as pd

df = pd.DataFrame({
    'col1': [1, 2, 3],
    'col2': [4, 5, 6]
})

# Rename specific columns
df_renamed = df.rename(columns={'col1': 'Column1', 'col2': 'Column2'})

# Rename in-place
df.rename(columns={'col1': 'Column1'}, inplace=True)
```

#### Method 2: Assigning to `columns` attribute

```
df.columns = ['Column1', 'Column2']
```

#### Method 3: Using `str` methods

```
# Convert all column names to lowercase
df.columns = df.columns.str.lower()

# Remove spaces from column names
df.columns = df.columns.str.replace(' ', '_')
```

## 13. What is the difference between Series and DataFrame in Pandas?

### Answer:

| Feature           | Series                     | DataFrame                                           |
|-------------------|----------------------------|-----------------------------------------------------|
| <b>Dimensions</b> | 1D (single column)         | 2D (multiple rows and columns)                      |
| <b>Structure</b>  | Labeled array              | Table/spreadsheet-like structure                    |
| <b>Axes</b>       | Only index                 | Index (rows) and columns                            |
| <b>Creation</b>   | <code>pd.Series()</code>   | <code>pd.DataFrame()</code>                         |
| <b>Use Case</b>   | Single variable analysis   | Multi-variable analysis                             |
| <b>Access</b>     | <code>series[index]</code> | <code>df[column]</code> or <code>df.loc[row]</code> |

### Examples:

```
import pandas as pd

# Creating a Series
series = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(series['a']) # Output: 10
```



```
# Creating a DataFrame
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
})
print(df['A']) # Returns a Series
print(df.loc[0]) # Returns a Series (row)
```

## 14. How do you calculate descriptive statistics for a DataFrame?

### Answer:

Descriptive statistics summarize the central tendency, dispersion, and shape of a dataset.

### Using describe():

```
import pandas as pd

df = pd.DataFrame({
    'Age': [25, 30, 35, 40, 45],
    'Salary': [50000, 60000, 75000, 80000, 95000]
})

# Get summary statistics
print(df.describe())
# Output includes: count, mean, std, min, 25%, 50%, 75%, max

# For specific columns
print(df[['Age']].describe())
```

### Individual statistics:

```
df.mean()          # Mean
df.median()        # Median
df.std()           # Standard deviation
df.var()           # Variance
df.min()           # Minimum
df.max()           # Maximum
df.sum()           # Sum
df.count()         # Count
df.quantile(0.25)  # 25th percentile
```

## 15. How do you reshape a NumPy array?

### Answer:

Reshaping changes the dimensions of an array without changing its data.

### Using reshape():

```
import numpy as np

# 1D array to 2D
arr = np.array([1, 2, 3, 4, 5, 6])
reshaped = arr.reshape(2, 3)
# Output:
# [[1 2 3]
#  [4 5 6]]

# 2D array to 3D
```

```
arr_2d = np.array([[1, 2, 3], [4, 5, 6]])
reshaped = arr_2d.reshape(2, 3, 1)

# Reshape with -1 (automatic dimension)
reshaped = arr.reshape(2, -1) # NumPy calculates the -1 dimension
```

### Using flatten() and ravel():

```
# Flatten to 1D
arr_2d = np.array([[1, 2, 3], [4, 5, 6]])
flattened = arr_2d.flatten() # Returns copy: [1 2 3 4 5 6]
raveled = arr_2d.ravel() # Returns view (more efficient)

# Difference: flatten() returns a copy, ravel() returns a view
```

## 16. How do you create a pivot table in Pandas?

### Answer:

A pivot table reorganizes data from rows to columns and applies aggregation functions.

### Syntax:

```
import pandas as pd

df = pd.DataFrame({
    'Date': ['2023-01-01', '2023-01-01', '2023-01-02', '2023-01-02'],
    'Product': ['A', 'B', 'A', 'B'],
    'Sales': [100, 200, 150, 250]
})

# Basic pivot table
pivot = pd.pivot_table(df,
                        values='Sales', # Values to aggregate
                        index='Date', # Row labels
                        columns='Product', # Column labels
                        aggfunc='sum') # Aggregation function

# Multiple aggregations
pivot = pd.pivot_table(df,
                        values='Sales',
                        index='Date',
                        columns='Product',
                        aggfunc=['sum', 'mean'])
```

### Output example:

| Product    | A   | B   |
|------------|-----|-----|
| Date       |     |     |
| 2023-01-01 | 100 | 200 |
| 2023-01-02 | 150 | 250 |

## 17. How do you calculate the correlation between variables in a DataFrame?

### Answer:

Correlation measures the linear relationship between two variables, ranging from -1 to 1.

### Calculating correlation:

```
import pandas as pd

df = pd.DataFrame({
    'Age': [25, 30, 35, 40, 45],
    'Salary': [50000, 60000, 75000, 80000, 95000]
})

# Correlation between two columns
corr = df['Age'].corr(df['Salary']) # Output: high positive correlation

# Correlation matrix (all numeric columns)
corr_matrix = df.corr()

# Correlation with a specific method
corr = df.corr(method='pearson') # Pearson (default), spearman, kendall
```

### Interpretation:

- **+1**: Perfect positive correlation
- **0**: No correlation
- **-1**: Perfect negative correlation

## 18. How do you find the maximum and minimum values in a NumPy array?

### Answer:

NumPy provides `max()` and `min()` functions for finding extreme values.

### Finding max/min:

```
import numpy as np

arr = np.array([3, 1, 4, 1, 5, 9, 2, 6])

# Maximum and minimum values
max_val = arr.max() # Output: 9
min_val = arr.min() # Output: 1

# 2D array operations
arr_2d = np.array([[1, 2, 3], [4, 5, 6]])

# Max/min of entire array
max_val = arr_2d.max() # Output: 6

# Max/min along axis
max_per_col = arr_2d.max(axis=0) # Output: [4 5 6]
max_per_row = arr_2d.max(axis=1) # Output: [3 6]

# Finding index of max/min
max_index = arr.argmax() # Output: 5 (index where 9 is located)
min_index = arr.argmin() # Output: 1 (first index where 1 is located)
```

## 19. How do you create a new column in a DataFrame that is a transformation of existing columns?

### Answer:

New columns can be created through various transformations of existing data.

#### Method 1: Direct calculation

```
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
})

# Create new column as sum of two columns
df['C'] = df['A'] + df['B']

# Create new column with scalar operation
df['D'] = df['A'] * 10
```

#### Method 2: Using apply() with lambda

```
# Create new column with conditional logic
df['Status'] = df['A'].apply(lambda x: 'High' if x > 2 else 'Low')

# Create new column using multiple columns
df['Ratio'] = df.apply(lambda row: row['A'] / row['B'], axis=1)
```

#### Method 3: Using np.where() for conditional assignment

```
import numpy as np

df['Category'] = np.where(df['A'] > 2, 'Above Average', 'Below Average')
```

#### Method 4: Using map() for categorical transformation

```
df['Grade'] = df['A'].map({1: 'F', 2: 'D', 3: 'C'})
```

## 20. How do you remove duplicate rows from a DataFrame?

### Answer:

The `drop_duplicates()` method removes duplicate rows from a DataFrame.

#### Basic usage:

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Alice', 'Charlie', 'Bob'],
    'Age': [25, 30, 25, 35, 30]
})

# Remove all duplicates (based on all columns)
df_clean = df.drop_duplicates()

# Remove duplicates based on specific column(s)
df_clean = df.drop_duplicates(subset=['Name'])
```

```
# Keep first occurrence (default)
df_clean = df.drop_duplicates(subset=['Name'], keep='first')

# Keep last occurrence
df_clean = df.drop_duplicates(subset=['Name'], keep='last')

# Remove all occurrences (keep=False)
df_clean = df.drop_duplicates(subset=['Name'], keep=False)
```

#### In-place removal:

```
df.drop_duplicates(inplace=True)
```

### Quick Reference Table

| Task              | Function                                    | Example                                      |
|-------------------|---------------------------------------------|----------------------------------------------|
| Read CSV          | <code>pd.read_csv()</code>                  | <code>df = pd.read_csv('file.csv')</code>    |
| Check shape       | <code>.shape</code>                         | <code>df.shape</code> → (rows, columns)      |
| First/last rows   | <code>.head()</code> , <code>.tail()</code> | <code>df.head(5)</code>                      |
| Data types        | <code>.dtypes</code>                        | <code>df.dtypes</code>                       |
| Missing values    | <code>.isnull().sum()</code>                | Counts NaN per column                        |
| Summary stats     | <code>.describe()</code>                    | Mean, std, min, max, etc.                    |
| Sort data         | <code>.sort_values()</code>                 | <code>df.sort_values('Age')</code>           |
| Filter rows       | <code>df[df['col'] &gt; value]</code>       | Boolean indexing                             |
| Group data        | <code>.groupby()</code>                     | <code>df.groupby('col').sum()</code>         |
| Merge DataFrames  | <code>pd.merge()</code>                     | Inner/left/right/outer joins                 |
| Create column     | <code>df['new'] = ...</code>                | <code>df['Total'] = df['A'] + df['B']</code> |
| Remove duplicates | <code>.drop_duplicates()</code>             | Removes duplicate rows                       |
| Replace values    | <code>.replace()</code>                     | <code>df.replace(old_val, new_val)</code>    |
| Export to CSV     | <code>.to_csv()</code>                      | <code>df.to_csv('output.csv')</code>         |

### Tips for Interview Success

1. **Practice with real datasets** - Use Kaggle or real business data to practice
2. **Know your libraries** - Be comfortable with Pandas, NumPy, and basic Matplotlib
3. **Understand SQL concepts** - Many interview questions bridge SQL and Python
4. **Write clean code** - Use meaningful variable names and add comments
5. **Explain your approach** - Walk through your reasoning when solving problems
6. **Test your code** - Always verify output with small datasets first
7. **Handle edge cases** - Consider missing values, duplicates, and data type mismatches
8. **Ask clarifying questions** - Understand requirements before writing code

9. **Use vectorized operations** - Avoid loops when possible for better performance
10. **Review your logic** - Double-check for off-by-one errors and incorrect assumptions